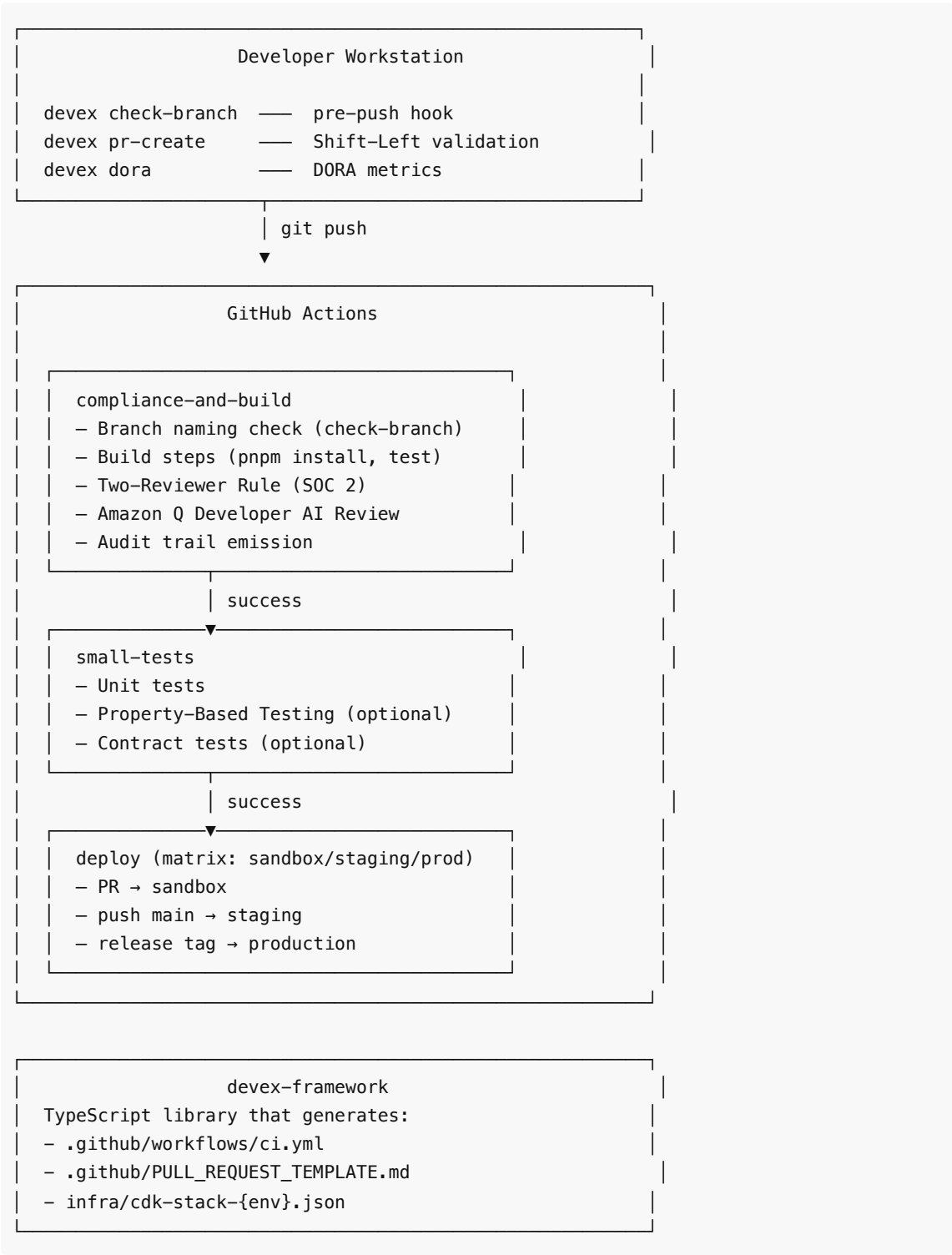


DevEx Golden Path — Design Document

Architecture Overview



Component Relationships

- `devex-framework` reads `devex.json` from the consumer project and generates CI/CD artifacts
- `devex-cli` reads the same `devex.json` for `validate` commands
- The generated CI workflow installs `devex-cli` via `cliSource` and runs `check-branch` during the pipeline
- `devex dora` queries GitHub API directly (not through the framework)
- Amazon Q Developer runs automated PR reviews during the compliance-and-build stage

Homologation Strategy

Phase	Gate	Who
Local	<code>devex validate</code> , pre-push hook	Developer
PR	Two-Reviewer Rule, compliance-and-build, Amazon Q AI Review	Peers + CI + AI
Sandbox	Automated deploy, manual smoke tests	QA
Staging	Automated deploy, integration tests	QA + PM
Production	Release tag, approval	Platform Team

Scalability

- **CLI:** Stateless, no external dependencies (except git). Scales horizontally by design.
- **Framework:** Pure code generation — no runtime overhead. Outputs static YAML/JSON.
- **CI:** Each consumer project has its own generated workflow. No shared infrastructure.
- **DORA:** Rate-limited by GitHub API (5000 req/h for authenticated users). The single `dora` command makes ~4 API calls per run.
- **Platform Team:** Framework is extensible via typed configuration. New features are additive — teams opt in by upgrading their `devex-framework` dependency. Custom pipeline stages are supported via the `buildSteps` array, avoiding platform-team bottlenecks.

Shift-Left Strategy

1. Pre-push hook (`devex init`) — catches branch naming and build failures before push
2. `devex pr-create` — runs validation locally and blocks PR creation if validation fails
3. CI `check-branch` — double-enforces naming convention in case hook was bypassed
4. Amazon Q Developer — automated AI code review on every PR
5. CI Two-Reviewer Rule — ensures peer review before deploy

This creates a defense-in-depth approach where issues are caught at the earliest possible stage.